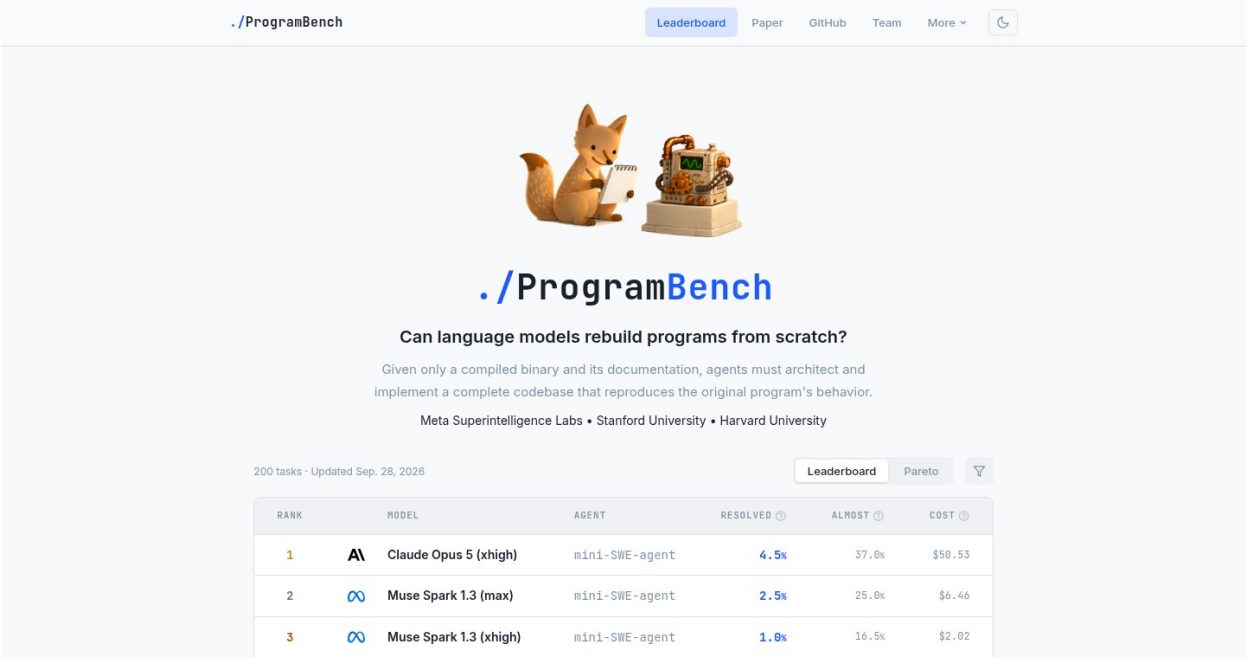
ProgramBench

実行ファイルの振る舞いから
コードベースを作り直す

sasakiy84

1. プログラムを再構築する課題
2. 課題の作成とデータの構成
3. 採点方法と到達水準
4. 評価での利用と入手方法

ProgramBench



[1] ProgramBench （2026-10-21 参照）
https://programbench.com

1. プログラムを再構築する課題

求めるのは、一つのプログラム全体

- 入力は、参照用の実行ファイルと使い方の文書。エージェントはその動作を調べる。
- 提出するのは、同じ動作を再現するソースコードとビルドスクリプト。実装言語とコードの分け方を自分で決める。
- 元のソースコードの復元ではなく、入出力やファイルの変化を再現する課題である。

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

実行して調べ、隠されたテストで評価する

- 参照プログラムには任意の入力を与え、標準出力・標準エラー・終了コード・生成ファイルを観察できる。
- 解答中はネット接続と逆コンパイルを使えず、元のソースコードや採点用テストは渡されない。
- 以下の実例は、公開配布された採点用テストから選んだ。単発の出力を答えるのではなく、こうした動作を持つプログラムを作る。

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

実例 1 ： ripgrep の文字列検索

公開テストに含まれる入力・操作

```
test.txt  の内容：
foo.*bar
[hello]
操作： ./executable -F "foo.*bar" test.txt
```

期待する動作

終了コードは 0 。出力に foo.*bar を含む。

-F は正規表現の記号を文字そのものとして扱う。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） burntsushi/ripgrep
https://huggingface.co/datasets/programbench/ProgramBench-Tests

実例 2 ： SQLite の問い合わせと JSON 出力

公開テストに含まれる入力・操作

```
./executable -json :memory: \
  "SELECT 1 as a, 'hello' as b;"
```

期待する動作

JSON 配列の要素は 1 件で、 a は 1 、 b は "hello" 。

SQL の計算だけでなく、指定された形式で結果を出す動作を再現する。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） sqlite/sqlite
https://huggingface.co/datasets/programbench/ProgramBench-Tests

実例３： xsv の CSV 行数

公開テストに含まれる入力・操作

```
test.csv の内容（\n は改行）：
name,age\nAlice,30\nBob,25\nCarol,35\n
操作： ./executable count test.csv
```

期待する動作

出力の前後の空白を除くと ３。標準エラーは空。

見出しをデータ行に数えないことまで、テストで確かめる。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） burntsushi/xsv
https://huggingface.co/datasets/programbench/ProgramBench-Tests

2. 課題の作成とデータの構成

既存のソフトウェアから課題を作る

- 単体の実行プログラムを作れる、コンパイル型言語のオープンソースプロジェクトを候補にする。
- エージェントがソースから参照プログラムをビルドし、その手順を保存する。
- 動作テストを作った後、ソース・テスト・実装を明かす生成物を除いて、解答環境を用意する。

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

200 課題は小さな道具から大規模システムまで

用途	含まれるプログラム
検索・データ処理	ripgrep 、 jq 、 xsv
圧縮・メディア処理	zstd 、 xz 、 FFmpeg
言語の実行	PHP 、 Lua 、 tinycc
データベース	SQLite 、 DuckDB

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

参照プログラムの言語構成

言語	課題数
Rust	107
Go	46
C	33
C++	12
Haskell / Java	各 1

既存プロジェクトの言語。解答には別の言語も使える。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照）
https://huggingface.co/datasets/programbench/ProgramBench-Tests

テスト生成と解答では、見える情報が違う

- テストを作るエージェントは、実行ファイルに加えてソース・既存テスト・文書も参照する。
- コードのどこを通ったかを測り、未確認の動作を追加で調べる。既存の動作テストも取り込む。
- 参照プログラムで安定して通るか、弱い判定や何もしないダミープログラムでも通るテストがないかを確認し、除外・修正する。

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

3. 採点方法と到達水準

完全解決率と、テスト通過率

- 完全解決率は、すべての採点テストに通った課題の割合。
- テスト通過率は、各課題で通ったテストの割合。途中までの進歩を見るためにも使う。
- 判定方法はテストごとに異なる。文字列の一致、 JSON の値、終了コード、生成ファイルなど、外から観察できる動作を確認する。

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

初期評価では、完全解決は 0 件

モデル	完全解決率	95% 以上通過
Claude Opus 4.7	0.0%	3.0%
Claude Opus 4.6	0.0%	2.5%
Gemini 3.1 Pro	0.0%	0.0%
GPT 5.4	0.0%	0.0%

200 課題、 mini-SWE-agent 。最大 1,000 ステップ・ 6 時間（ 2026 年公表）。
右列は 95% 以上通る課題の割合。

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

2026 年 9 月には、長文脈の評価にも採用

- Claude Fable 5.1 の評価では、参照実行ファイルのテスト通過率が 90% 未満の 34 課題を除き、 166 課題を使った。
- 残った課題でも、参照実行ファイルが通るテストだけを採点し、 6 時間の制限を外した。
- この条件での隠しテスト通過率は、 Fable 5.1 が 87.6% 、 Fable 5 が 86.3% 、 Opus 5 が 85.4% 。

[1] Anthropic (2026), Claude Fable 5.1 & Claude Mythos 5.1 System Card
https://www.anthropic.com/claude-fable-and-mythos-5-1

公式の公開結果では、完全解決も現れた

モデル	完全解決率	95% 以上通過
Claude Opus 5 (xhigh)	4.5%	37.0%
Muse Spark 1.3 (max)	2.5%	25.0%
GPT-5.6 Sol (xhigh)	1.0%	15.5%

2026-10-21 参照。 200 課題、 mini-SWE-agent 。右列は 95% 以上通る課題の割合。

[1] ProgramBench （ 2026-10-21 参照）
https://programbench.com

4. 評価での利用と入手方法

一つの課題にも、多数の動作テストがある

- 200 課題に合計 248,853 件のテスト関数がある。検索や変換の基本機能に加え、引数・出力形式・例外時の動作まで調べる。
- 各プロジェクトの難しさは大きく異なる。小さな CLI の操作と、 FFmpeg や言語処理系の再実装を同じ枠組みで評価する。
- 参照プログラムの動作を調べ、実装・実行・修正を繰り返すため、長い作業履歴を扱う。

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

配布先と、そこで手に入るもの

配布先	公開内容
公式サイト	課題の紹介・結果・論文への入口
GitHub	評価用コード・課題の情報・利用ガイド
Hugging Face	生成した動作テスト・帰属情報・ライセンス
Docker Hub	推論と評価に使う環境

[1] Meta FAIR (2026), ProgramBench （2026-10-21 参照）
https://github.com/facebookresearch/ProgramBench

評価環境を用意して、コードベースを提出する

- 公式ガイドは Linux の x86_64 環境を案内している。課題用のコンテナには、参照実行ファイルと文書がある。
- 解答中のネット接続を止め、完成したコードベースをアーカイブにして提出する。
- 評価用のコマンドが、課題ごとのテストを取得して実行し、通過・失敗と集計結果を出す。

[1] ProgramBench (2026), Usage Guide （ 2026-10-21 参照）
https://github.com/facebookresearch/ProgramBench/blob/main/docs/README.md

コードとテストの利用条件を確かめる

- ProgramBench の評価用コードは MIT ライセンスで公開されている。
- テストは AI が生成し、元のプロジェクトのテストに由来する部分を含みうる。各課題の帰属情報とライセンスが付いている。
- 再配布や改変では、使う課題の ATTRIBUTION.md と LICENSE を確認する。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照）
https://huggingface.co/datasets/programbench/ProgramBench-Tests

ProgramBench が測るもの

- 実行ファイルと文書だけを手がかりに、同じ動作をするコードベースをゼロから作る。
- 200 の既存ソフトウェアと、外部動作を確かめる多数のテストにより、プログラム全体を実装する能力を測る。
- 完全解決、部分的な通過、作業の進め方を見ることで、長時間のソフトウェア開発を評価できる。

[1] Yang et al. (2026), ProgramBench
https://arxiv.org/abs/2605.03546

Appendix

1. 異なる動作の追加例
2. 評価環境と利用条件
3. 参考文献

追加例 1 ： sd の空パターン置換

公開テストに含まれる入力・操作

```
標準入力： abc
操作： ./executable "" "X"
```

期待する動作

終了コードは 0 。出力に XaXbXcX を含む。

空の検索パターンは、各文字の前後の位置に一致する。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） chmln/sd
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 2 ： DuckDB の出力モード

公開テストに含まれる入力・操作

```
操作： ./executable --batch --no-init \
  -cmd ".mode csv"
標準入力： SELECT 42 as x;
```

期待する動作

出力に x と 42 を含み、罫線の ┌ は含まない。
終了コードは 0 。

先に指定した CSV 出力の設定を、標準入力の SQL にも適用する。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） duckdb/duckdb
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例３： zstd の圧縮と復元

公開テストに含まれる入力・操作

```
標準入力： "Hello, World! "  を 100 回
1.  ./executable -c  で圧縮
2.  圧縮結果を ./executable -dc  に渡す
```

期待する動作

復元後のバイト列が、入力と完全に一致する。

圧縮ファイルの見た目だけでなく、復元可能かを確認する。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） facebook/zstd
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 4 ： FFmpeg へ標準入力で動画を渡す

公開テストに含まれる入力・操作

```
入力： 配布テストの 2 秒の test.mp4 のバイト列
操作： ./executable -i - -f null -
```

期待する動作

終了コードは 0 。標準エラーに
Input #0 と Stream #0:0 を含む。

ファイルパスの代わりに - を指定して、動画を標準入力から読ませる。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） ffmpeg/ffmpeg
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 5 ： jq の名前付き変数

公開テストに含まれる入力・操作

```
標準入力： {}
操作： ./executable --arg foo bar '$foo'
```

期待する動作

出力の前後の空白を除くと "bar" 。終了コードは 0 。

--arg は変数名と値の二つの引数を消費し、その変数をフィルターで参照できる。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） jqlang/jq
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 6 ： PHP のスクリプト引数

公開テストに含まれる入力・操作

```
./executable -r 'echo json_encode($argv);' \
  -- test 123
```

期待する動作

終了コードは 0 。出力に "test" と "123" を含む。

-- より後の文字列を、実行する PHP コードから引数として読める。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） php/php-src
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 7 ： yj で YAML を JSON へ変換する

公開テストに含まれる入力・操作

```
標準入力（\n は改行）：
key: value\nnumber: 42
操作： ./executable
```

期待する動作

出力に "key" 、 "value" 、 "number" 、 42 を含む。
終了コードは 0 。標準エラーは空。

引数を省いたときにも、既定の YAML から JSON への変換を行う。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） sclevine/yj
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 8 ： fd の探索の深さ

公開テストに含まれる入力・操作

```
探索先： file.txt 、 sub/file2.txt 、
         sub/deep/file3.txt
操作： ./executable --max-depth 1
```

期待する動作

出力に file.txt と sub を含み、
file2.txt と deep は含まない。

一つ下の階層までを調べ、より深い場所へ進まない。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） sharkdp/fd
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 9 ： hexyl の 16 進数表示

公開テストに含まれる入力・操作

```
標準入力のバイト： 00 10 20 FF （16 進数）
操作： ./executable --base=hexadecimal \
  --color=never
```

期待する動作

終了コードは 0 。出力に 00 、 10 、 20 、 ff を含む。

表示の基数と色の有無を、引数で切り替える。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） sharkdp/hexyl
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 10 ： gron の JSON 展開

公開テストに含まれる入力・操作

```
標準入力（JSON）：
{"string":"hello","number":42,"float":3.14,
 "bool":true,"null_val":null}
操作： ./executable -m
```

期待する動作

各値を代入文へ展開し、参照結果と一致する。

数値・文字列・真偽値・ null の型を保つ。

[1] ProgramBench (2026), Generated Tests （2026-10-21 参照） tomnomnom/gron
https://huggingface.co/datasets/programbench/ProgramBench-Tests

追加例 10 ： gron の期待出力

参照テキスト（全文）

```
json = {};
json.bool = true;
json.float = 3.14;
json.null_val = null;
json.number = 42;
json.string = "hello";
```

出力全体をこのテキストと比較する。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照） tomnomnom/gron
https://huggingface.co/datasets/programbench/ProgramBench-Tests

動作を調べる環境と、採点環境

- 推論用コンテナでは参照プログラムの実行だけを許し、ソースやビルド成果物へのアクセスを除く。
- 提出されたコードを別の評価用環境でビルドし、参照プログラムに対して作られたテストを実行する。
- 配布テストには、個別に採点から除いたものも記録される。採点では、除外されたテストを通過率の分母から外す。

[1] ProgramBench (2026), Usage Guide （2026-10-21 参照）
https://github.com/facebookresearch/ProgramBench/blob/main/docs/README.md

掲載例に付属する元プロジェクトの条件

配布物のライセンス表示	今回扱うプロジェクト
Unlicense / Public-Domain	ripgrep 、 xsv / SQLite
MIT	sd 、 DuckDB 、 jq 、 gron
Apache-2.0	yj 、 fd 、 hexyl
BSD-3-Clause / LGPL-2.1	zstd 、 PHP / FFmpeg

公式テスト配布の各課題に、帰属情報とライセンス本文が付属する。

[1] ProgramBench (2026), Generated Tests （ 2026-10-21 参照）
https://huggingface.co/datasets/programbench/ProgramBench-Tests

参考文献

ProgramBench: Can Language Models Rebuild Programs From Scratch?

Yang et al. (2026), arXiv:2605.03546

課題の定義、作成工程、初期評価、コード構造の分析を報告する。

https://arxiv.org/abs/2605.03546

ProgramBench Usage Guide

Meta FAIR (2026), GitHub

推論用環境、提出物、採点と集計の手順を案内する。

https://github.com/facebookresearch/ProgramBench/blob/main/docs/README.md

参考文献

ProgramBench Generated Tests

ProgramBench (2026), Hugging Face

プログラムごとの動作テストと、元プロジェクトへの帰属・利用条件を公開する。

https://huggingface.co/datasets/programbench/ProgramBench-Tests

System Card: Claude Fable 5.1 & Claude Mythos 5.1

Anthropic (2026), System Card

長文脈・複数エージェントの評価における、 ProgramBench の使用条件と結果を報告する。

https://www.anthropic.com/claude-fable-and-mythos-5-1